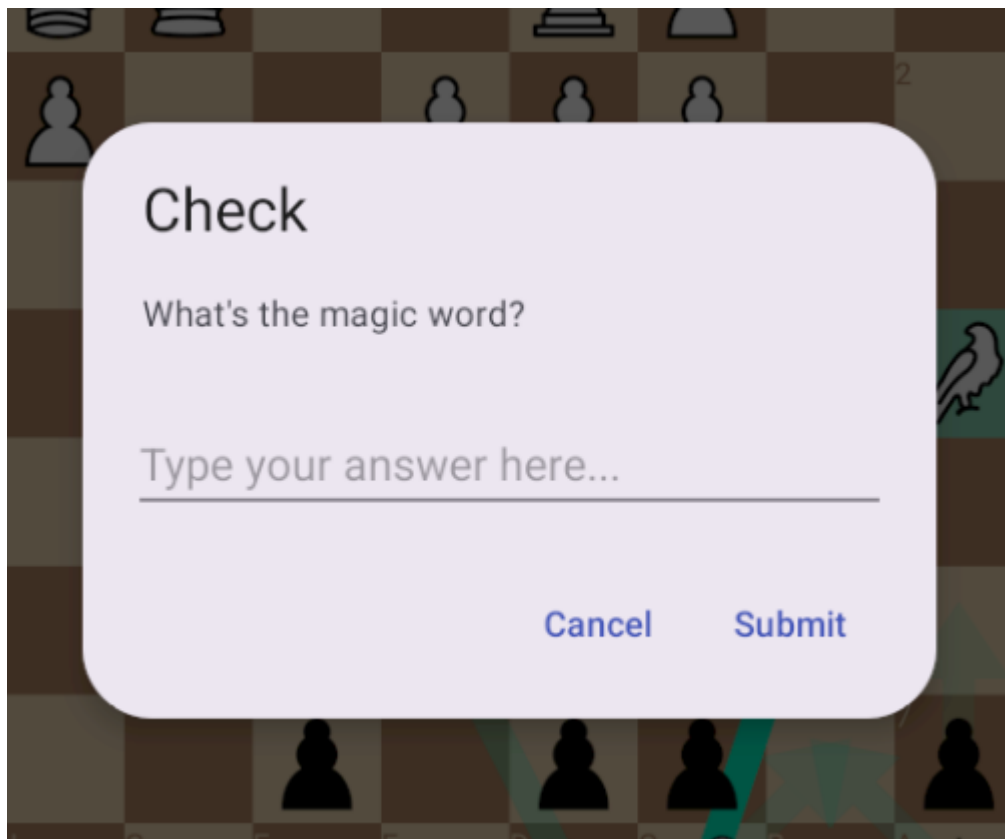


In `GameFragment`. It shows if we trigger hints over 10 times, it will ask us for a magic word. It calls `verifyPalyerInput` to check the word.



```
private final void requestHintsIfEnabled() {
    if (this.hintsEnabled) {
        this.numberOfHints++;
        requestHints();
        if (this.numberOfHints > 10) {
            Context contextRequireContext = requireContext();
            Intrinsics.checkNotNullExpressionValue(contextRequireContext, "requireContext(...)");
            showQueryDialog(contextRequireContext, new Function1() { // from class: de.tadris.flang.ui.fragment.GameFragment$$ExternalSyntheticLambda9
                @Override // kotlin.jvm.functions.Function1
                public final Object invoke(Object obj) {
                    return GameFragment.requestHintsIfEnabled$lambda$19(this.f$0, (String) obj);
                }
            });
            this.numberOfHints = 0;
        }
    }
}

/* JADX INFO: Access modifiers changed from: private */
public static final Unit requestHintsIfEnabled$lambda$19(GameFragment gameFragment, String response) {
    Intrinsics.checkNotNullParameter(response, "response");
    Context contextRequireContext = gameFragment.requireContext();
    Intrinsics.checkNotNull(contextRequireContext);
    Toast.makeText(gameFragment.requireContext(), ComputerHintsHelper.verifyPalyerInput(contextRequireContext, response), 1).show();
    return Unit.INSTANCE;
}
```

`verifyPalyerInput` calls `processHintInContext`. It reads data from `background_pattern.png` and xor `bytes[i % 1en(bytes)]`. `bytes` is the bytes of `JUMP_ITEM`.

```
public final class ComputerHintsHelper {
    private static final String JUMP_ITEM = "aDifferentProcess_ohDear";
    public static final ComputerHintsHelper INSTANCE = new ComputerHintsHelper();
    private static final char[] ASSET_NAME = {'b', 'a', 'c', 'k', 'g', 'r', 'o', AbstractJsonLexerKt.UNICODE_ESC, 'n', 'd', '_', 'p', 'a', 't', 't', 'e', 'r', 'n', ' ', 'p', ' ', 'n', 'g'};

    private ComputerHintsHelper() {}

    private final byte[] processHintInContext(Context context, String assetName) throws Exception {
        InputStream inputStreamOpen = context.getAssets().open(assetName);
        try {
            InputStream inputStream = inputStreamOpen;
            int iAvailable = inputStream.available();
            byte[] bArr = new byte[iAvailable];
            if (inputStream.read(bArr) > 0) {
                byte[] bArr2 = new byte[iAvailable];
                byte[] bytes = JUMP_ITEM.getBytes(Charsets.UTF_8);
                Intrinsics.checkNotNullExpressionValue(bytes, "getBytes(...)");
                for (int i = 0; i < iAvailable; i++) {
                    bArr2[i] = (byte) (bArr[i] ^ bytes[i % bytes.length]);
                }
                CloseableKt.closeFinally(inputStreamOpen, null);
                return bArr2;
            }
        }
        Unit unit = Unit.INSTANCE;
        CloseableKt.closeFinally(inputStreamOpen, null);
        return new byte[0];
    }
}
```

I wrote a script to see what is the output. It shows the type `dex` at the beginning.

```
1 data = open("background_pattern.png", "rb").read()
2
3 jump_item = b"aDifferentProcess_ohDear"
4
5 tmp = []
6 for i in range(len(data)):
7     tmp.append(chr(data[i] ^ jump_item[i % len(jump_item)]))
8
9 print(tmp)
```

[illegible]

Modify the script, to make the `.dex` file.

```
1 data = open("background_pattern.png", "rb").read()
2
3 jump_item = b"aDifferentProcess_ohDear"
4
5 tmp = []
6 for i in range(len(data)):
7     tmp.append(data[i] ^ jump_item[i % len(jump_item)])
8
9 dex = bytes(tmp)
10 open("test.dex", "wb").write(dex)
```

Open `test.dex` in Jadx, there is only one class `HiddenCompiledDexClass`.

`LetsJumpAgain` is just like Caesar cipher.

```
protected static String letsJumpAgain(String text, int shift) {
    StringBuilder result = new StringBuilder();
    for (char character : text.toCharArray()) {
        if (Character.isLetter(character)) {
            char base = Character.isUpperCase(character) ? 'A' : 'a';
            character = (char) (((character - base) + shift) + 26) % 26 + base);
        }
        result.append(character);
    }
    return result.toString();
}
```

`mergeThemTogether` is xor again like `processHintInContext`.

```
protected static byte[] mergeThemTogether(byte[] inputData, char[] password) {
    byte[] output = new byte[inputData.length];
    for (int i = 0; i < inputData.length; i++) {
        char keyChar = password[i % password.length];
        output[i] = (byte) (inputData[i] ^ keyChar);
    }
    return output;
}
```

The logic of the check is simple. The magic word is just `valueCheck`.

`letsJumpAgain(jumpPlus, -6) = Clever`. The entire magic word is

`thisIsThePassword_C1ever_no?`.

```
public class HiddenCompiledDexClass {
    private static final String keepGoing = "ThePassword";
    private static final char separator = '_';
    private static final char[] jumpPlus = {'I', 'r', 'k', 'b', 'k', 'x'};
    private static final char[] arrayOfCharsAgainWithATwist = {'s', 'I'};
    private static final char[] arrayOfChars = {'t', 'h', 'i', 's'};
    private static final char[] slightChangeAgain = {'?', 'o', 'n'};
    private static final byte[] whatAreThese = {50, 36, 40, 52, 50, 27, 61, 12, 12, 62, 6, 44, 23, 22, 27, 19, 59, 54, 45, 51, 4, 24, 58, 23, 49, 13, 0, 91, 17, 12, 54, 23, 44, 11, 11, 11, 9, 49, 18, 0, 44, 35, 34, 15};

    public static String checkParameter(String parameter) {
        String valueCheck = new String(arrayOfChars) + arrayOfCharsAgainWithATwist[1] + arrayOfCharsAgainWithATwist[0] + "ThePassword_" + letsJumpAgain(new String(jumpPlus), -6) + "_" + slightChangeAgain[2] + slightChangeAgain[1] + slightChangeAgain[0];
        if (parameter.equals(valueCheck)) {
            return new String(mergeThemTogether(whatAreThese, parameter.toCharArray()));
        }
        return "\"Game over, man! Game over!\" - Hicks, Aliens";
    }
}
```



We can also merge ourselves. I wrote a script:

```

1  whatAreThese = [50, 36, 40, 52, 50, 27, 61, 12, 12, 62, 6, 44,
2    23, 22, 27, 19, 59, 54, 45, 51, 4, 24, 58, 23, 49, 13, 0, 91,
3    17, 12, 54, 23, 44, 11, 11, 11, 9, 49, 18, 0, 44, 35, 34, 15]
4  password = b"thisIsThePassword_Clever_no?"
5  flag = ""
6  for i in range(len(whatAreThese)):
7      flag += chr(whatAreThese[i] ^ password[i % len(password)])
8  print(flag)

```

```

(mahiro@kali) - [~/Homework/defense/hint]
$ python3 merge.py
FLAG{hiding_data_in_an_encoded_dex_class_TM}

```

FLAG{hiding_data_in_an_encoded_dex_class_TM}